ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÁO CÁO TIẾN ĐỘ BÀI TẬP LỚN – LẦN 2

MÔN: ĐIỆN TOÁN ĐÁM MÂY

Mã lớp: INT3319 2

ĐỀ TÀI

SPOTPOOL – NỀN TẢNG CHO THUÊ TÀI NGUYÊN TÍNH TOÁN THEO JOB, ĐA NGƯỜI DÙNG, TÍNH PHÍ THEO MỨC SỬ DỤNG TRÊN AWS

Giảng viên hướng dẫn: TS. Hoàng Xuân Tùng

Nhóm thực hiện: G1

STT	Họ và tên	MSSV
1	Phạm Minh Nghĩa (Nhóm trưởng)	24021584
2	Nguyễn Minh Khuê	24021536
3	Đoàn Ngọc Anh	24021363

Hà Nội, 10/2026

MỤC LỤC

Chương 1. Tổng quan về dự án 2
1.1. Giới thiệu chung về tính năng 2
1.2. Người dùng của dự án 3
1.3. Quy trình sử dụng sản phẩm 4
1.4. Ví dụ sử dụng 6
Chương 2. Giải pháp thực hiện (chính thức) 8
2.1. Kiến trúc tính năng 9
2.2. Kiến trúc triển khai 12
2.3. Các chi tiết khác của giải pháp 18
Chương 3. Kế hoạch triển khai và ước tính nỗ lực 21
3.1. Bảng hạng mục công việc 22
3.2. Tiến độ theo tuần 25
Tài liệu tham khảo 26

CHƯƠNG 1. TỔNG QUAN VỀ DỰ ÁN

Ghi chú: Không thay đổi so với Báo cáo 1. Nội dung chương được giữ nguyên để tiện theo dõi.

1.1. Giới thiệu chung về tính năng

SpotPool là một nền tảng cho thuê sức tính toán theo job, xây dựng trên AWS theo mô hình PaaS. Người dùng không thuê nguyên một máy chủ. Họ nộp một công việc cần chạy (job), ví dụ nén một loạt video hay chạy một script xử lý dữ liệu, chọn gói dịch vụ, rồi nhận kết quả và hóa đơn. Hệ thống tự chọn máy, tự bật thêm hoặc tắt bớt máy theo lượng việc, và chỉ tính tiền theo thời gian job chạy thực tế. Người dùng không cần tài khoản AWS, không cần thẻ quốc tế và không cần hiểu các khái niệm như IAM, VPC hay Auto Scaling.

Nhóm không chỉ sử dụng cloud mà xây dựng một nền tảng cloud thu nhỏ ở mức PaaS trên hạ tầng AWS, tự hiện thực đủ năm đặc trưng theo định nghĩa của NIST: tự phục vụ theo nhu cầu (người dùng tự nộp job, tự nạp credit, không cần quản trị viên duyệt), truy cập rộng rãi qua mạng (cổng web, API và dòng lệnh), gộp tài nguyên (nhiều người dùng chung một cụm máy), co giãn nhanh (cụm máy tự tăng giảm theo lượng job, gói Tiết kiệm về 0 máy khi hết việc) và dịch vụ đo lường được (đo vCPU-giây và lập hóa đơn cho từng người). Để giảm chi phí, gói Tiết kiệm chạy trên máy Spot của AWS. Khi máy Spot bị thu hồi, job được chạy tiếp trên máy khác: job nén video chỉ phải làm lại video đang dở, còn job script chạy lại từ đầu. Job bị thu hồi quá 2 lần sẽ được chuyển sang máy On-Demand. Chi phí của SpotPool và của EC2 On-Demand tự thuê được đo trên hóa đơn AWS thật; so sánh với Fargate và AWS Batch được tính từ bảng giá.

Nền tảng cung cấp hai gói dịch vụ, khác nhau ở loại máy và đơn giá (Bảng 1.1).

Bảng 1.1. Hai gói dịch vụ của SpotPool

Đặc điểm	Ổn định	Tiết kiệm
Loại máy	On-Demand, luôn giữ sẵn 1 máy trong giờ hoạt động	Spot, hết việc thì về 0 máy
Mục tiêu thời gian chờ đến khi job bắt đầu chạy (SLA, áp dụng cho 95% số job)	≤ 1 phút	≤ 15 phút
Khi máy bị thu hồi	Không áp dụng	Job video làm lại video đang dở, job script chạy lại từ đầu; quá 2 lần thì chuyển sang On-Demand
Đơn giá	Cao	Thấp

Mọi gói dùng chung một đơn vị tài nguyên gọi là **suất**: 1 vCPU và 1,5 GB RAM. Mỗi máy 2 vCPU chạy cùng lúc 2 suất, có thể của hai người dùng khác nhau; job nặng có thể xin suất đôi (2 vCPU, 3 GB). Tiền được tính theo vCPU-giây được cấp (số vCPU của suất × số giây chạy), không theo mức CPU mà job dùng thật.

Các mục tiêu SLA ở trên là cam kết thiết kế; nhóm sẽ đo thời gian chờ thật ở tuần 4 và công bố cùng hóa đơn AWS. Đơn giá mỗi suất sẽ được tính từ chi phí máy thật trên hóa đơn AWS, chia theo hệ số sử dụng của cụm, cộng phần dự phòng cho job chạy lại và biên lợi nhuận; số cụ thể được chốt sau khi đo ở tuần 3–4. Tiền trên nền tảng là credit ảo cấp cho người dùng; thanh toán thật nằm ngoài phạm vi môn học.

So với AWS Batch, dịch vụ chạy job có sẵn của AWS, SpotPool có thêm lớp nhà cung cấp mà AWS Batch không có: người dùng không cần tài khoản AWS, chỉ cần đăng ký bằng email, chọn gói, đặt trần chi phí và nhận hóa đơn riêng tính theo vCPU-giây. Phần chạy job bên dưới (hàng đợi, tự co giãn, xử lý máy Spot bị thu hồi) nhóm tự xây thay vì dùng AWS Batch để hiểu và đo được từng cơ chế.

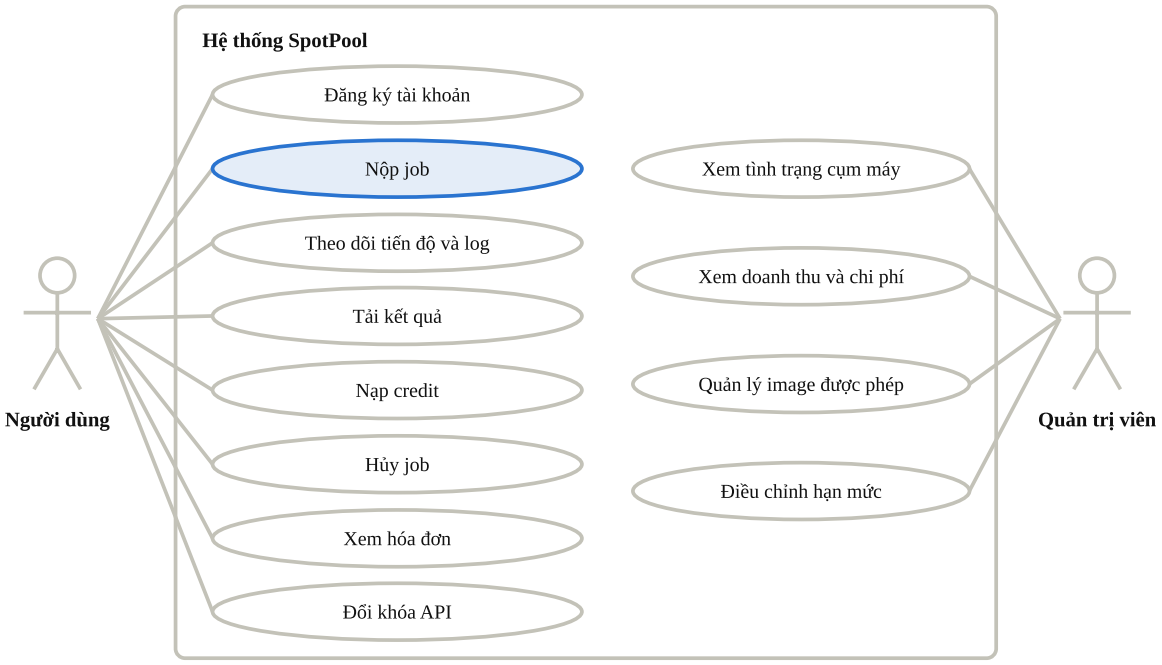
1.2. Người dùng của dự án

SpotPool hướng tới các nhóm người dùng có nhu cầu thuê sức tính toán linh hoạt. Bảng 1.2 mô tả chi tiết ba nhóm đối tượng chính.

Bảng 1.2. Các nhóm người dùng của SpotPool

Nhóm người dùng	Ví dụ	Nhu cầu	Khó khăn khi tự dùng AWS
Sinh viên, học viên	Làm đồ án xử lý ảnh, video, dữ liệu	Thỉnh thoảng chạy job dài hoặc nhiều việc song song mà không phải treo máy cá nhân	Không có thẻ quốc tế, sợ hóa đơn bất ngờ, quota tài khoản mới thấp
Nhóm nghiên cứu nhỏ	Chạy mô phỏng tính toán; huấn luyện mô hình học máy truyền thống; tìm siêu tham số song song	Chạy nhiều job theo đợt, chi phí thấp, chấp nhận chờ	Phải tự dựng và quản lý cụm máy, khó xử lý Spot bị thu hồi
Nhà phát triển, startup nhỏ	Chuyển đổi video cho ứng dụng, xử lý báo cáo định kỳ	Gọi qua API, biết trước chi phí tối đa	Không muốn tốn công vận hành hạ tầng, máy hay bị nhàn rỗi

Nền tảng chỉ dùng máy CPU nên không nhắm tới huấn luyện học sâu cần GPU. Người thành thạo AWS có thể tự dùng AWS Batch với giá gốc tương đương. SpotPool nhắm đến người dùng nhỏ cần sự đơn giản, tương tự cách Heroku hay Vercel được xây dựng trên AWS.



Hình 1.1. Biểu đồ use case của SpotPool

Người dùng làm mọi việc qua web hoặc API mà không cần quản trị viên duyệt. Khi AWS thu hồi máy Spot, hệ thống tự xử lý để job chạy tiếp mà không ai phải thao tác, nên bước này không có trong biểu đồ mà được thể hiện ở Hình 1.2.

1.3. Quy trình sử dụng sản phẩm

Người dùng đạt mục đích "chạy được job với chi phí biết trước" qua năm quy trình dưới đây. Hình 1.2 ở cuối mục minh họa vòng đời của một job.

Quy trình 1 – Đăng ký và nhận credit

1. Người dùng đăng ký bằng email trên cổng web. Hệ thống sinh một khóa API và chỉ hiện một lần để người dùng lưu lại; khóa này dùng để đăng nhập cổng web.
2. Hệ thống tạo hồ sơ, cấp số dư credit ban đầu và hạn mức mặc định (số job chạy đồng thời, số job lỗi miễn phí mỗi ngày).
3. Người dùng dùng chính khóa API đó để gọi từ dòng lệnh hoặc từ ứng dụng của họ, và có thể đổi khóa mới khi lộ khóa cũ.

Quy trình 2 – Nộp job

1. Chọn loại job: nén video (mẫu có sẵn) hoặc chạy script của người dùng trong một image thuộc danh sách được phép.
2. Tải dữ liệu đầu vào lên. File đi thẳng vào kho lưu trữ qua đường dẫn tải lên có thời hạn, không qua máy chủ trung gian.
3. Chọn gói Ổn định hoặc Tiết kiệm. Với job script chạy lâu, hệ thống nhắc rằng ở gói Tiết kiệm, nếu máy bị thu hồi thì script phải chạy lại từ đầu.

4. Đặt **trần chi phí** (số credit tối đa). Hệ thống quy đổi ra thời gian chạy tối đa theo đơn giá gói. Với mẫu nén video, hệ thống gợi ý mức chi phí dựa trên lịch sử chạy; với script tự viết, người dùng có thể chạy thử trên một phần nhỏ dữ liệu trước.
5. Bấm Chạy. Hệ thống kiểm tra số dư và hạn mức rồi đưa job vào hàng đợi của gói đã chọn.

Quy trình 3 – Theo dõi và nhận kết quả

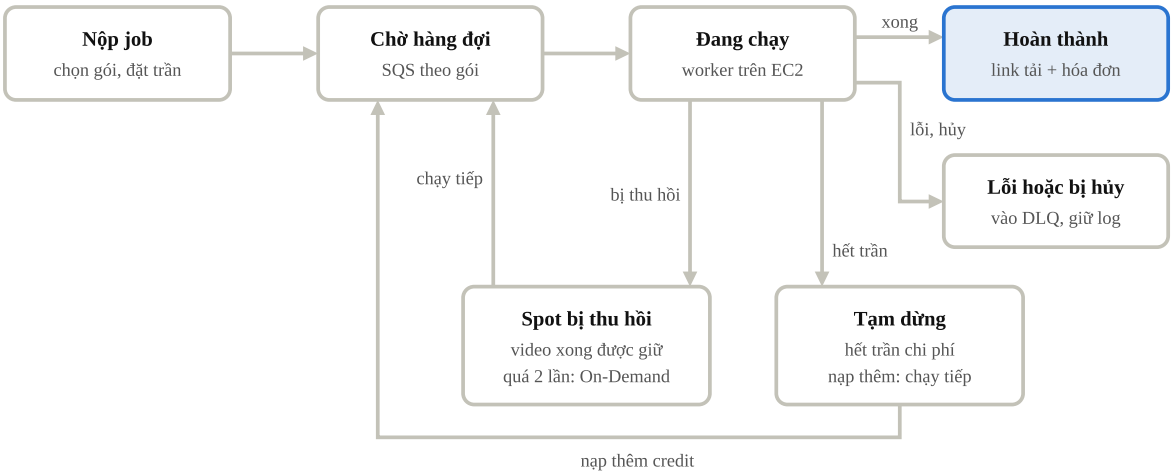
1. Trang chi tiết job hiển thị trạng thái theo thời gian thực: đang chờ, đang chạy, bị gián đoạn và chạy tiếp, hoàn thành.
2. Người dùng xem log và tiến độ (với job video: số video đã xong).
3. Khi hoàn thành, kết quả được giữ 7 ngày để người dùng tải về lúc nào cũng được (mỗi lần bấm tải, hệ thống cấp một link tạm thời), kèm chi phí thực tế của job.

Quy trình 4 – Xử lý khi có sự cố

- *Máy Spot bị thu hồi:* hệ thống tự xử lý, người dùng chỉ thấy trạng thái "bị gián đoạn, đang chạy tiếp". Phần chạy lại do nền tảng chịu, không tính vào hóa đơn.
- *Hết trần chi phí:* job tạm dừng. Với job video, các video đã xong vẫn được giữ, người dùng nạp thêm để chạy nốt phần còn lại; với job script, người dùng chỉ trả phần đã chạy.
- *Job lỗi hoặc bị hủy:* miễn phí trong hạn mức số lần mỗi ngày. Người dùng vẫn tải được log và kết quả dở dang.

Quy trình 5 – Xem hóa đơn và quản trị

1. Người dùng xem hóa đơn theo từng job: gói, số vCPU-giây, đơn giá, thành tiền, cùng biểu đồ mức dùng theo ngày.
2. Quản trị viên xem trang tổng quan: số máy đang chạy theo loại (Spot, On-Demand) và vùng sẵn sàng (AZ), số job chờ trong từng hàng đợi, doanh thu từ người dùng so với chi phí trả cho AWS.



Hình 1.2. Vòng đời của một job

Job bị thu hồi quay về hàng đợi, các video đã xong được giữ lại; job hết trần tạm dừng chờ nạp thêm. Chỉ job lỗi hoặc bị hủy mới rời luồng chính.

1.4. Ví dụ sử dụng

Ví dụ sau dùng số liệu giả định để minh họa cả quy trình; đơn giá thật sẽ được đặt sau khi đo chi phí.

1. **Nhu cầu.** Lan, sinh viên năm 3, cần nén 20 video bài giảng (mỗi video 10 phút) xuống 720p để nộp lên hệ thống học tập của trường. Việc không gấp, chỉ cần xong trước 8 giờ sáng hôm sau.
2. **Nộp job.** Lan chọn mẫu "Nén video", tải 20 file lên, chọn gói Tiết kiệm vì việc không gấp. Dựa trên lịch sử chạy, hệ thống ước lượng job cần khoảng 200 suất-phút, tức khoảng 1 suất-phút cho mỗi phút video. Với đơn giá giả định 50 đ/suất-phút, chi phí dự kiến khoảng 10.000 đ; Lan đặt trần 15.000 đ.
3. **Hệ thống xử lý.** Mỗi video là một việc riêng, 20 việc được đưa vào hàng đợi Tiết kiệm. Cụm Tiết kiệm đang ở 0 máy nên tự bật 4 máy Spot, tức 8 suất, nén 8 video cùng lúc.
4. **Sự cố.** Giữa chừng, AWS thu hồi một máy. Hai video đang nén dở trên máy đó quay lại hàng đợi và được máy khác nén lại từ đầu; các video đã xong không bị ảnh hưởng. Lan chỉ thấy trạng thái "bị gián đoạn, đang chạy tiếp".
5. **Kết quả.** Khoảng 30 phút sau, cả 20 video xong. Lan nhận link tải và hóa đơn khoảng 10.000 đ; phần chạy lại do máy bị thu hồi không bị tính tiền.
6. **Giải phóng tài nguyên.** Hết việc, cụm Tiết kiệm tự giảm về 0 máy, nền tảng không tốn thêm tiền máy.

Ví dụ này đi qua đủ năm đặc trưng cloud của NIST: Lan tự phục vụ qua mạng, các máy được dùng chung với người dùng khác, cụm máy co giãn từ 0 lên 4 rồi về 0, và mức dùng được đo để tính đúng phần đã chạy.

CHƯƠNG 2. GIẢI PHÁP THỰC HIỆN (CHÍNH THỨC)

Giải pháp chính thức giữ kiến trúc ba lớp và bộ dịch vụ AWS của Báo cáo 1. Bảng 2.0 tổng hợp 12 điểm thay đổi; các mục 2.1–2.2 đã được cập nhật theo bảng này, và mục 2.3 mới chốt các quy ước dùng chung giữa các thành phần.

Bảng 2.0. Tổng hợp những thay đổi so với Báo cáo 1

STT	Thay đổi	Lý do
1	Thêm bảng DynamoDB `Tasks` (khóa `jobId` + `taskId`), lưu trạng thái từng việc, số lần bị thu hồi (`interruptCount`), lần chạy (`attempt`), worker đang giữ (`owner`), hạn giữ (`leaseUntil`), nhịp tim.	Một job gồm nhiều việc (mỗi video một việc); cần theo dõi và giành từng việc riêng.
2	Bảng `Users` tách số dư thành `creditAvailable` và `creditHeld`.	Điều kiện của DynamoDB không làm được phép trừ; giữ credit chỉ cần điều kiện `creditAvailable >= costCap`, đảm bảo nguyên tử.
3	Khóa bảng `Usage` là `userId` + `{jobId}#{taskId}#{attempt}`.	Khóa xác định trước nên ghi lại nhiều lần cũng không tính tiền hai lần.
4	Thêm bước "Bấm Chạy" `POST /jobs/{id}/start`: API kiểm tra đủ file, giữ credit, chuyển job sang `QUEUED`, rồi mới ghi `submissions/{userId}/{jobId}/manifest.json`. S3 Event kích hoạt dispatcher khi manifest xuất hiện, không phải khi từng file được tải lên.	Đúng Quy trình 2 bước 5 ở Chương 1; dispatcher chỉ chạy khi job đã được duyệt. Link tải lên chỉ ghi được vào `uploads/` nên người dùng không giả được manifest.
5	Thêm endpoint `POST /users`, `POST /users/me/rotate-key`, `POST /jobs/{id}/start`, `GET /jobs`, `POST /credits`, `GET /admin/overview`.	Phủ đủ các use case ở Hình 1.1 (đăng ký, đổi khóa, nạp credit, quản trị).
6	Message SQS bỏ "thời gian tối đa" và "số lần bị thu hồi"; thêm `schemaVersion`; message không đổi suốt vòng đời.	Số lần thu hồi lưu ở `Tasks`. Thời gian tối đa do worker tính từ ngân sách còn lại lúc bắt đầu chạy, vì người dùng có thể nạp thêm.
7	Worker ghi kết thúc bằng một giao dịch `TransactWriteItems` trên `Tasks`, `Jobs` và `Usage`.	Tránh cộng trùng mức dùng khi SQS giao trùng hoặc worker chạy lại.
8	Job có bộ đếm `tasksRemaining`; job kết thúc khi bộ đếm về 0. Trạng thái "bị gián đoạn" không lưu riêng mà suy ra từ `interruptCount > 0`.	Job không bị treo khi có việc lỗi; tránh hai worker ghi đè trạng thái.

STT	Thay đổi	Lý do
9	Thêm Lambda xử lý `sqs-dlq` (đánh dấu việc `FAILED` sau 10 lần giao) và Lambda billing chạy theo DynamoDB Stream của `Jobs`.	Báo cáo 1 có DLQ và Lambda billing nhưng chưa nói cơ chế kích hoạt.
10	Cổng web: React tĩnh trong S3 bucket private, phân phối qua Amazon CloudFront (HTTPS, tên miền mặc định `*.cloudfront.net`, Origin Access Control).	S3 static website hosting chỉ có HTTP; CloudFront cho HTTPS và giữ bucket private.
11	Máy worker cài Docker và nạp worker agent bằng user data khi khởi động (khoảng 1,5 phút), thay cho Custom AMI.	Đơn giản hơn khi code worker còn thay đổi; mục tiêu ≤ 1 phút của gói Ổn định vẫn giữ được nhờ 1 máy chạy sẵn.
12	Chốt API Gateway kiểu REST với Lambda authorizer kiểu TOKEN (giữ như Báo cáo 1, ghi rõ không dùng HTTP API).	Authorizer đọc `authorizationToken`, `methodArn` và các Lambda đọc `requestContext.authorizer.userId`; chỉ REST API cung cấp đúng định dạng này.

2.1. Kiến trúc tính năng

Hệ thống chia thành ba lớp: lớp truy cập (người dùng tương tác), lớp dịch vụ nhà cung cấp (quản lý khách hàng, xếp job, tính phí, xử lý job lỗi) và lớp thực thi (chạy job trên cụm máy dùng chung). Hình 2.1 thể hiện các khối chức năng và hướng dữ liệu giữa chúng.



Hình 2.1. Kiến trúc tính năng theo ba lớp

Yêu cầu đi từ trên xuống qua API và hàng đợi; trạng thái và mức dùng đi ngược lên để tính phí và hiển thị. So với Báo cáo 1, lớp dịch vụ có thêm khối Xử lý job lỗi (DLQ).

Lớp truy cập. Cổng web cho người dùng (đăng ký, nộp job, theo dõi, hóa đơn), trang quản trị cho nhóm vận hành, và API/dòng lệnh cho người dùng kỹ thuật. Cả ba dùng chung một bộ API nên mọi thao tác trên web đều làm được bằng API.

Lớp dịch vụ nhà cung cấp gồm sáu khối:

- *Tài khoản và hạn mức:* xác thực người dùng bằng khóa API, giới hạn số job đồng thời và số job lỗi miễn phí. Số dư credit tách thành hai trường: `creditAvailable` (dùng được) và `creditHeld` (đang giữ cho các job đã bấm Chạy). Giữ credit là một lệnh UpdateItem có điều kiện `creditAvailable >= costCap`, chuyển `costCap` từ `creditAvailable` sang `creditHeld` trong cùng một lần ghi, nên hai yêu cầu đồng thời không thể cùng tiêu một khoản credit.
- *Tiếp nhận job và báo giá:* kiểm tra image có trong danh sách cho phép, quy đổi trần chi phí ra thời gian chạy tối đa để báo cho người dùng, cấp đường dẫn tải lên chỉ ghi được vào `uploads/`. Khi người dùng bấm Chạy (`POST /jobs/{id}/start`), khối này kiểm tra đã đủ file, giữ credit, chuyển job sang

`QUEUED` rồi mới ghi `submissions/{userId}/{jobId}/manifest.json`. Vì link tải lên không ghi được vào `submissions/`, người dùng không thể tự tạo manifest để bỏ qua bước kiểm tra.

- *Xếp job theo gói:* Lambda dispatcher chỉ chạy khi manifest xuất hiện. Dispatcher tách job video thành nhiều việc, mỗi video một việc (job script là một việc), tạo một bản ghi `Tasks` cho mỗi việc rồi gửi message vào hàng đợi đúng gói; job mang bộ đếm `tasksRemaining` bằng số việc. Việc bị thu hồi quá 2 lần (theo `interruptCount`) được chuyển sang hàng đợi On-Demand. Việc bị kẹt khi máy sập nguồn hay đứt mạng được xử lý mà không cần bộ lập lịch riêng: worker gửi nhịp tim lên bản ghi `Tasks` mỗi 30 giây và gia hạn `leaseUntil` thêm 120 giây; khi worker ngừng hoạt động, message tự hiện lại trên hàng đợi sau visibility timeout, và worker khác được nhận lại việc khi leaseUntil đã qua. Lượt chạy bị gián đoạn không tính tiền.
- *Đo mức dùng và tính phí:* mỗi lần chạy được tính tiền của một việc có đúng một bản ghi `Usage`, khóa `userId` + `{jobId}#{taskId}#{attempt}`; khóa xác định trước nên ghi lại nhiều lần vẫn chỉ là một bản ghi. Lambda billing được kích hoạt bởi DynamoDB Stream của bảng `Jobs` khi bản ghi job thay đổi ([13]); nó cộng vCPU-giây được cấp cho các phần chạy thành công, trừ credit, trả phần credit đang giữ chưa dùng về `creditAvailable` và áp dụng chính sách miễn phí job lỗi. Khối này cũng ghi lại "chi phí rủi ro" nền tảng phải gánh.
- *Xử lý job lỗi (DLQ)* (khối mới): message bị giao quá 10 lần mà chưa xong ([11]) được SQS chuyển sang `sqs-dlq`. Lambda DLQ đọc hàng đợi này, đánh dấu việc `FAILED` và giảm `tasksRemaining` của job, nên job vẫn kết thúc thay vì bị treo vì một việc lỗi. Log và kết quả dở dang vẫn được giữ để người dùng tải.
- *Giám sát và báo cáo chi phí:* thu số liệu hàng đợi, số máy, số lần thu hồi và chi phí AWS để hiển thị trên dashboard và phục vụ bài toán hòa vốn.

Lớp thực thi gồm bốn khối:

- *Hàng đợi theo gói:* hai hàng đợi Ổn định và Tiết kiệm, cùng một hàng đợi job lỗi (DLQ). Các worker cùng kéo việc từ hàng đợi (competing consumers), nên tải được cân bằng tự nhiên mà không cần bộ cân bằng tải. Vì SQS Standard có thể giao một message nhiều lần, worker phải nhận việc trước khi chạy bằng một lệnh UpdateItem có điều kiện trên bảng `Tasks`, chỉ thành công khi việc đang chờ hoặc đã quá `leaseUntil`. Nếu việc đã có máy khác nhận, Worker giành hụt thì bỏ message đó. Message giữ nguyên suốt vòng đời và có trường `schemaVersion` để worker nhận biết định dạng.

- *Cụm worker:* mỗi gói một nhóm máy 2 vCPU, mỗi máy chạy cùng lúc 2 suất. Máy cài Docker và nạp worker agent bằng user data khi khởi động. Trên mỗi máy, worker agent lấy việc và chạy việc trong container bị giới hạn đúng bằng suất được cấp, không có mạng ra ngoài, với thời gian tối đa tính từ ngân sách còn lại của job lúc bắt đầu chạy. Khi việc kết thúc, worker ghi kết quả lên S3 rồi ghi trạng thái việc, bộ đếm của job và mức dùng trong một giao dịch `TransactWriteItems` trên ba bảng `Tasks`, `Jobs`, `Usage`: cả ba cùng được ghi hoặc không bảng nào thay đổi ([12]), nên SQS giao trùng hay worker chạy lại cũng không cộng mức dùng hai lần. Vì job của hai người dùng có thể chạy chung một máy, các giới hạn container giữ cho job này không làm chậm và không đọc được dữ liệu của job kia.
- *Tự co giãn:* chia số job đang chờ cho số máy đang chạy (backlog-per-instance, cách AWS khuyến nghị khi co giãn theo hàng đợi SQS) để tăng giảm số máy; có luật riêng để bật máy đầu tiên khi nhóm đang ở 0 máy; nhóm Ổn định giữ sẵn 1 máy trong giờ hoạt động.
- *Chịu lỗi khi Spot bị thu hồi:* worker hỏi cảnh báo thu hồi của AWS mỗi 5 giây (AWS báo trước khoảng 2 phút). Khi nhận cảnh báo, worker ngừng nhận việc mới, tăng `interruptCount` của việc đang dở trong `Tasks` và trả việc về hàng đợi ngay. Job nén video được chia theo từng video (task chunking): mỗi video là một việc độc lập, nên máy bị thu hồi thì chỉ phải làm lại video đang dở. Job script chạy lại từ đầu; việc bị thu hồi quá 2 lần được chuyển sang máy On-Demand bằng chính message cũ, vì số lần thu hồi được đọc từ `Tasks` chứ không nằm trong message. Theo AWS Spot Instance Advisor, tần suất thu hồi Spot trung bình trong lịch sử dưới 5% (tính theo 30 ngày gần nhất); nhóm vẫn thiết kế chịu lỗi vì tỷ lệ này thay đổi theo thời điểm, và dùng AWS FIS để chủ động giả lập thu hồi khi kiểm thử vì trong 5 tuần rất có thể không gặp lần thu hồi thật nào.

2.2. Kiến trúc triển khai

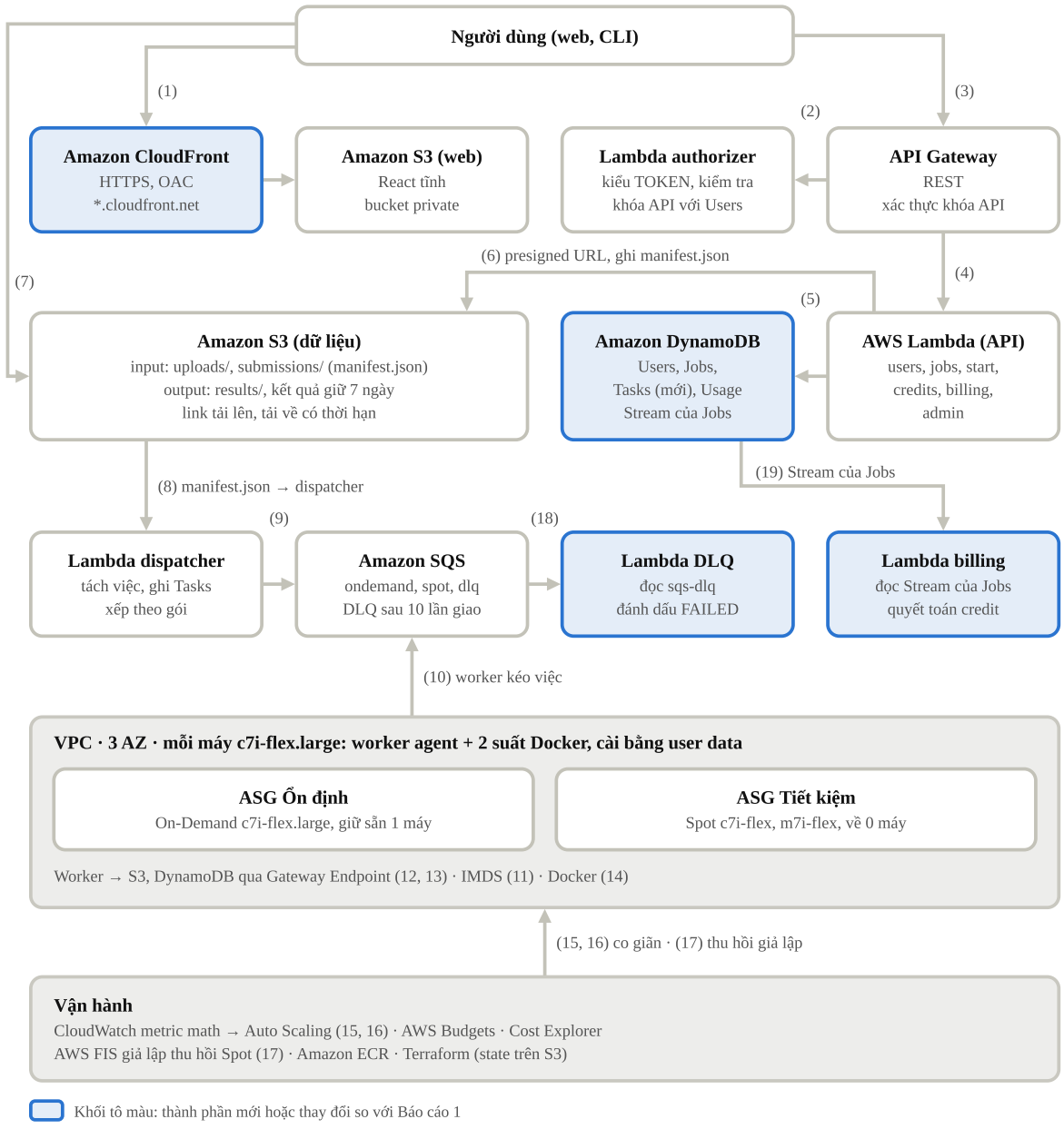
Toàn bộ hệ thống chạy trong một tài khoản AWS, một region (ap-southeast-1 ở Singapore, gần Việt Nam), trải trên cả 3 vùng sẵn sàng (AZ) của region, và được dựng hoàn toàn bằng Terraform (state lưu trên S3). Lớp truy cập và lớp dịch vụ dùng dịch vụ serverless (không cần duy trì máy chạy liên tục); chỉ lớp thực thi dùng máy EC2. Hình 2.2 thể hiện các thành phần triển khai.

Bảng 2.1 ánh xạ từng khối chức năng sang dịch vụ AWS triển khai.

Bảng 2.1. Ánh xạ khối chức năng sang dịch vụ AWS

Khối chức năng	Dịch vụ AWS triển khai
Cổng web, trang quản trị	Ứng dụng web tĩnh (React) đặt trong S3 bucket private, phân phối qua Amazon CloudFront: HTTPS với tên miền mặc định `*.cloudfront.net`, CloudFront đọc bucket bằng Origin Access Control
API, dòng lệnh	Amazon API Gateway kiểu REST (không dùng HTTP API) + AWS Lambda
Tài khoản và hạn mức	Lambda authorizer kiểu TOKEN của API Gateway, so khóa API với mã băm lưu trong bảng `Users` trên Amazon DynamoDB; số dư tách thành `creditAvailable` và `creditHeld`
Tiếp nhận job, báo giá	Các Lambda tạo job và bấm Chạy, bảng `Jobs` trên DynamoDB, S3 bucket input (`uploads/` cho dữ liệu, `submissions/` cho manifest)
Theo dõi từng việc	Bảng `Tasks` trên DynamoDB (khóa `jobId` + `taskId`)
Xếp job theo gói	Lambda dispatcher, kích hoạt bởi S3 Event trên `submissions/*/manifest.json`
Hàng đợi theo gói	Amazon SQS: `sqs-ondemand`, `sqs-spot`, `sqs-dlq`
Xử lý job lỗi	Lambda DLQ, đọc `sqs-dlq` qua event source mapping
Cụm worker	2 Auto Scaling Group gồm máy EC2 2 vCPU; máy cài Docker và nạp worker agent bằng user data khi khởi động, image lưu trên Amazon ECR. Gói Ổn định dùng On-Demand `c7i-flex.large`; gói Tiết kiệm dùng Spot với chiến lược price-capacity-optimized. Tài khoản đang ở gói Free của AWS (200 USD credit, chi phí trừ dần vào credit) nên chỉ dùng được một số loại máy như `c7i-flex.large`, `m7i-flex.large`; nếu nâng lên gói Paid, nhóm mở rộng ra khoảng 10 loại máy x86 2 vCPU theo khuyến nghị của AWS. Khi đo so sánh chi phí, nhóm cố định `c7i-flex.large` để kết quả công bằng
Tự co giãn	Target tracking trên chỉ số backlog-per-instance tính bằng CloudWatch metric math (số job chờ ÷ số máy đang chạy), không cần hàm tự đẩy chỉ số; một CloudWatch alarm bật máy đầu tiên khi nhóm đang ở 0; scheduled scaling giữ 1 máy Ổn định trong giờ hoạt động
Kết quả	S3 bucket output (`results/{userId}/{jobId}/{taskId}/{attempt}/`), lifecycle rule tự xóa sau 7 ngày
Đo mức dùng, tính phí	Bảng `Usage` trên DynamoDB (khóa `userId` + `{jobId}#{taskId}#{attempt}`), Lambda billing kích hoạt bởi DynamoDB Stream của bảng `Jobs`
Mạng	VPC trải 3 AZ, public subnet, S3 Gateway Endpoint và DynamoDB Gateway Endpoint

Khối chức năng	Dịch vụ AWS triển khai
Giám sát, chi phí, kiểm thử lỗi	CloudWatch dashboard và log, AWS Budgets, Cost Explorer với cost allocation tag, AWS FIS
Hạ tầng như mã	Terraform, state lưu trên S3



Hình 2.2. Kiến trúc triển khai trên AWS

Dữ liệu lớn đi thẳng giữa người dùng, worker và S3; Lambda chỉ trao đổi thông tin điều khiển nhỏ qua DynamoDB và SQS.

Mạng và chi phí. Worker đặt trong public subnet của cả 3 AZ, chặn toàn bộ kết nối đi vào bằng security group. Worker đọc ghi S3 và DynamoDB qua Gateway Endpoint (miễn phí) và không dùng NAT Gateway. Mỗi máy có một địa chỉ IPv4 công khai để gọi SQS, ECR và CloudWatch; từ tháng 2/2024, AWS tính 0,005 USD/giờ cho mỗi địa

chỉ này ([6]). Nhóm chấp nhận khoản này vì rẻ hơn NAT Gateway hay VPC Interface Endpoint khi cụm chỉ có vài máy. Truyền dữ liệu giữa EC2 và S3 trong cùng region không tính phí, nên rải máy Spot ra nhiều AZ để tăng khả năng có máy mà không tốn thêm phí truyền.

Cổng web qua CloudFront. Ứng dụng React được build thành file tĩnh và đặt trong một S3 bucket private. CloudFront phân phối ứng dụng qua HTTPS với tên miền mặc định `*.cloudfront.net`, nên nhóm không phải mua tên miền hay tự quản lý chứng chỉ. CloudFront đọc bucket bằng Origin Access Control, nên bucket không phải mở công khai ([9]). Báo cáo 1 dự kiến mở web trực tiếp từ S3, nhưng endpoint website của S3 không hỗ trợ HTTPS ([10]).

API Gateway kiểu REST. Nhóm giữ API Gateway kiểu REST như Báo cáo 1 và ghi rõ không dùng HTTP API. Lambda authorizer kiểu TOKEN nhận `authorizationToken` (khóa API trong header) và `methodArn`, trả về policy kèm `userId`; các Lambda phía sau đọc `userId` ở `requestContext.authorizer.userId`. Chỉ REST API cung cấp đúng định dạng sự kiện này, nên đổi sang HTTP API sẽ phải viết lại authorizer và mọi Lambda.

Cách ly người dùng. Chỉ image trong ECR thuộc danh sách cho phép mới được chạy; danh sách gồm hai image `spotpool-video` và `spotpool-script`. Mỗi container chạy không có mạng, với hệ thống file chỉ đọc, không có Linux capability nào, dưới user không đặc quyền, và bị giới hạn CPU, RAM, thời gian chạy (chi tiết ở mục 2.3). Dữ liệu vào ra được worker agent chuẩn bị sẵn trong thư mục riêng. Worker agent chỉ được cấp quyền IAM tối thiểu cần cho hàng đợi, bucket và bảng của mình.

Khởi động máy và giữ SLA. Máy worker không dùng Custom AMI như dự kiến ở Báo cáo 1. Launch template chứa user data cài Docker và nạp worker agent khi máy khởi động, mất khoảng 1,5 phút. Cách này đơn giản hơn khi code worker còn thay đổi: sửa worker agent không phải tạo lại AMI. Tính cả thời gian Auto Scaling phát hiện tải, bật một máy mới mất vài phút, nên mục tiêu ≤ 1 phút của gói Ổn định được giữ bằng 1 máy On-Demand chạy sẵn trong giờ hoạt động.

Bảng 2.2 liệt kê giao diện tương tác giữa các thành phần; số thứ tự trùng với số trong ngoặc ở Hình 2.2. So với Báo cáo 1, các dòng 1, 2, 3, 4, 6, 8, 9, 12, 13, 14 được cập nhật; dòng 18, 19 là giao diện mới.

Bảng 2.2. Giao diện tương tác giữa các thành phần

STT	Từ → Đến	Giao diện, giao thức	Nội dung trao đổi
1	Trình duyệt → CloudFront → S3 (web)	HTTPS qua CloudFront; CloudFront đọc bucket private bằng Origin Access Control	Tải ứng dụng web tĩnh
2	API Gateway → Lambda authorizer	Authorizer kiểu TOKEN, gọi trước khi chuyển yêu cầu, có lưu đệm kết quả	`authorizationToken` (khóa API trong header) và `methodArn`; trả về policy kèm `userId` nếu khóa hợp lệ
3	Web, CLI → API Gateway	REST/JSON qua HTTPS, header `Authorization: Bearer <khóa API>` (trừ `POST /users`)	`POST /users`, `POST /users/me/rotate-key`, `POST /jobs`, `GET /jobs`, `GET /jobs/{id}`, `POST /jobs/{id}/start`, `POST /jobs/{id}/cancel`, `POST /jobs/{id}/topup`, `POST /credits`, `GET /billing`, `GET /admin/overview`
4	API Gateway → Lambda	Lambda proxy integration của REST API	Yêu cầu đã xác thực; Lambda đọc `userId` ở `requestContext.authorizer.userId`
5	Lambda → DynamoDB	AWS SDK: PutItem, Query, UpdateItem có điều kiện	Tạo job, tạm giữ credit, kiểm tra hạn mức một cách nguyên tử
6	Lambda → S3	AWS SDK: tạo presigned URL, PutObject	Link tải đầu vào (PUT, chỉ vào `uploads/`) và link tải kết quả (GET) có thời hạn; ghi `manifest.json` khi bấm Chạy
7	Người dùng → S3	HTTPS PUT/GET theo presigned URL	Dữ liệu đầu vào, kết quả
8	S3 input → Lambda dispatcher	S3 Event Notification (ObjectCreated) trên `submissions/*/manifest.json`	Báo job đã được duyệt và đủ dữ liệu; dispatcher bắt đầu tách việc và xếp hàng
9	Lambda → SQS	SendMessage	Message mỗi việc: `schemaVersion`, `jobId`, `taskId`, `userId`, `tier`, `kind`, `vcpu`, `image`, `params`, `input`, `output`; không đổi suốt vòng đời
10	Worker → SQS	ReceiveMessage (long polling), ChangeMessageVisibility, SendMessage, DeleteMessage	Nhận việc; gia hạn thời gian giữ việc khi chạy lâu; đặt về 0 để trả việc khi bị thu hồi; chuyển sang `sqs-ondemand` khi bị thu hồi quá 2 lần; xóa khi xong
11	Worker → Instance Metadata (IMDSv2)	HTTP nội bộ, hỏi mỗi 5 giây	Phát hiện cảnh báo thu hồi Spot (`spot/instance-action`)

STT	Từ → Đến	Giao diện, giao thức	Nội dung trao đổi
12	Worker → S3 qua Gateway Endpoint	AWS SDK	Tải đầu vào; ghi kết quả vào `results/{userId}/{jobId}/{taskId}/{attempt}/`
13	Worker → DynamoDB qua Gateway Endpoint	AWS SDK: UpdateItem có điều kiện, TransactWriteItems	Giành việc trên `Tasks` (đang chờ, hoặc đã quá `leaseUntil`); nhịp tim, tiến độ; ghi kết thúc bằng một giao dịch trên `Tasks`, `Jobs`, `Usage`
14	Worker agent → Docker	Docker Engine API qua Unix socket	Chạy container theo suất: `--cpus <vcpu>`, `--memory <vcpu×1536>m`, `--network none`, giới hạn thời gian (đủ các cờ ở mục 2.3)
15	SQS, Auto Scaling → CloudWatch	Chỉ số có sẵn, CloudWatch metric math	Số job chờ (`ApproximateNumberOfMessagesVisible`) chia số máy đang chạy (`GroupInServiceInstances`) ra backlog-per-instance
16	CloudWatch → Auto Scaling	Target tracking, CloudWatch alarm	Tăng giảm số máy mỗi nhóm; bật máy đầu tiên khi đang ở 0
17	AWS FIS → EC2 Spot	Thí nghiệm `aws:ec2:send-spot-instance-interruptions`	Giả lập thu hồi máy Spot khi kiểm thử và demo
18	`sqs-dlq` → Lambda DLQ	Event source mapping	Message bị giao quá 10 lần; Lambda đánh dấu việc `FAILED` trên Tasks và giảm `tasksRemaining` trên Jobs trong một giao dịch
19	DynamoDB Stream (`Jobs`) → Lambda billing	DynamoDB Streams, event source mapping	Bản ghi job thay đổi (ví dụ job kết thúc); Lambda đọc Usage của job rồi quyết toán credit bằng một giao dịch trên Users và Jobs

Luồng một job qua các giao diện. Đây là luồng thiết kế. Ở mốc M1, nhóm đã chạy thực tế trên AWS các bước tạo job, tải lên, bấm Chạy, dispatcher, worker và xem trạng thái bằng cách gọi trực tiếp các Lambda (chưa qua API Gateway và authorizer). Phần giữ credit ở bước 3 và quyết toán credit ở bước 6 chưa làm, thuộc hạng mục 5 và 14.

1. Người dùng gọi `POST /jobs` (3, 4); Lambda tạo bản ghi job ở trạng thái `UPLOADING` (5) và trả về link tải lên (6).
2. Người dùng tải file thẳng lên `uploads/` theo link (7).
3. Người dùng gọi `POST /jobs/{id}/start`; Lambda kiểm tra đủ file, giữ credit, chuyển job sang `QUEUED` (5) rồi ghi `manifest.json` (6).
4. S3 Event kích hoạt dispatcher (8); dispatcher tạo các bản ghi `Tasks` và gửi một message cho mỗi việc (9).

5. Worker kéo message (10), giành việc trên `Tasks` (13), tải đầu vào (12), chạy container (14), ghi kết quả (12), ghi kết thúc bằng một giao dịch (13) rồi xóa message (10).
6. Bản ghi job thay đổi kích hoạt Lambda billing quyết toán credit (19); người dùng xem trạng thái và lấy link tải kết quả qua API (3, 6, 7).

2.3. Các chi tiết khác của giải pháp

Các quy ước dưới đây được ghi trong tài liệu hợp đồng giao diện `contracts.md` (hạng mục 22) để ba thành viên làm song song mà vẫn ghép được.

Trạng thái job và việc. Job và việc dùng chung một bộ trạng thái, riêng `UPLOADING` chỉ có ở job (Bảng 2.3).

Bảng 2.3. Trạng thái job và việc

Trạng thái	Job	Việc	Hiển thị cho người dùng
`UPLOADING`	Có	Không	Đang tải dữ liệu lên, chưa bấm Chạy
`QUEUED`	Có	Có	Đang chờ
`RUNNING`	Có	Có	Đang chạy
`PAUSED`	Có	Có	Tạm dừng do hết trần chi phí, chờ nạp thêm
`SUCCEEDED`	Có	Có	Hoàn thành
`FAILED`	Có	Có	Lỗi
`CANCELLED`	Có	Có	Đã hủy

Job kết thúc khi `tasksRemaining` về 0. Lúc đó job thành `SUCCEEDED` nếu không có việc nào lỗi (`tasksFailed = 0`), ngược lại thành `FAILED`; bên nào (worker hoặc Lambda DLQ) giảm bộ đếm về 0 sẽ đặt trạng thái cuối bằng một lệnh ghi có điều kiện. Các video đã xong vẫn tải được và vẫn tính tiền; phần lỗi được miễn phí trong hạn mức job lỗi mỗi ngày. Trạng thái "bị gián đoạn, đang chạy tiếp" ở Quy trình 4 không lưu riêng mà suy ra khi có việc có `interruptCount > 0`, nên không có hai worker cùng ghi đè một trường trạng thái của job.

Các tham số thời gian và giới hạn được chốt như Bảng 2.4.

Bảng 2.4. Tham số vận hành

Tham số	Giá trị	Dùng ở
Chu kỳ nhịp tim	30 giây	Worker cập nhật bản ghi `Tasks`
Hạn giữ việc (lease)	120 giây	Quá `leaseUntil`, worker khác được giành lại việc
Chu kỳ hỏi cảnh báo thu hồi	5 giây	Worker → IMDSv2

Tham số	Giá trị	Dùng ở
Số lần giao tối đa trước DLQ	10	`sqs-ondemand`, `sqs-spot` → `sqs-dlq`
Số lần thu hồi tối đa trên Spot	2	Quá 2 lần thì chuyển sang `sqs-ondemand`
Thời gian giữ kết quả	7 ngày	Lifecycle rule của bucket output
Thời gian khởi động máy worker	khoảng 1,5 phút	User data cài Docker và worker agent

Loại máy worker. Gói Ổn định dùng `c7i-flex.large` (2 vCPU, 4 GB RAM), đủ chạy cùng lúc 2 suất 1,5 GB và còn khoảng 1 GB cho hệ điều hành, Docker và worker agent. Gói Tiết kiệm dùng thêm `m7i-flex.large` (2 vCPU, 8 GB RAM) để có thêm nguồn máy Spot; mỗi máy vẫn chỉ chạy 2 suất. Nhóm đã kiểm tra cả hai loại đều thuộc gói Free của tài khoản và đều hỗ trợ Spot ở cả 3 AZ (ap-southeast-1a, 1b, 1c).

Container chạy việc. Worker agent chạy mỗi việc với các cờ sau:

```
--network none --read-only --cap-drop ALL --security-opt no-new-
privileges --pids-limit 64 --user 65534:65534 --cpus <vcpu> --memory
<vcpu×1536>m --memory-swap <vcpu×1536>m
```

Container không có mạng, hệ thống file chỉ đọc, không giữ Linux capability nào, không cho leo quyền, tối đa 64 tiến trình, không dùng swap và chạy bằng user không đặc quyền 65534:65534 (nobody). CPU và RAM đúng bằng suất: 1 vCPU được 1536 MB, suất đôi 2 vCPU được 3072 MB. Container đọc đầu vào ở `/input/payload` và ghi kết quả vào `/output/result`; worker agent gắn file đầu vào ở chế độ chỉ đọc vào `/input/payload`, gắn một thư mục ghi được vào `/output` và cấp `/tmp` dạng tmpfs 64 MB (`--tmpfs /tmp:rw,noexec,nosuid,size=64m`), rồi đẩy kết quả lên `results/{userId}/{jobId}/{taskId}/{attempt}/`. Mỗi lần chạy có thư mục riêng theo `attempt`, nên lần chạy lại không ghi đè kết quả của lần trước.

Image. Danh sách cho phép gồm hai image: `spotpool-video` (mẫu nén video bằng FFmpeg) và `spotpool-script` (chạy script Python của người dùng). Image script đã có trên ECR, image video đang làm (hạng mục 8).

Đơn vị tiền. Credit là số nguyên; đơn giá tính theo credit/vCPU-giờ, tạm đặt 6000 cho gói Ổn định và 3000 cho gói Tiết kiệm, chốt sau khi đo ở tuần 3–4. Ví dụ, một suất 1 vCPU chạy 10 phút ở gói Tiết kiệm tốn 3000 × 1 × 10/60 = 500 credit. Mức dùng chỉ cộng vào job khi mỗi việc kết thúc, nên các việc chạy song song có thể làm job vượt trần, tối đa khoảng một việc trên mỗi suất đang chạy. Khi worker thấy hết ngân sách, job chuyển `PAUSED` và các việc đang chạy dừng ở nhịp tim kế tiếp. Người dùng không

bao giờ bị tính quá `costCap`; phần vượt do nền tảng chịu và được tính vào chi phí rủi ro.

Quản lý hạ tầng bằng mã. Toàn bộ hạ tầng dựng bằng Terraform, state lưu trên S3, nên có thể dựng lại hoặc xóa toàn bộ môi trường bằng một lệnh.

CHƯƠNG 3. KẾ HOẠCH TRIỂN KHAI VÀ ƯỚC TÍNH NỖ LỰC

Đến ngày 25/10/2026, nhóm đạt mốc M1 đúng kế hoạch (cuối tuần 2, minh chứng): luồng chính chạy thực tế trên AWS từ nộp job đến tải kết quả bằng cách gọi trực tiếp các Lambda, chưa qua API Gateway; cổng web chưa có. 7 hạng mục đã hoàn thành (1, 2, 3, 4, 6, 22, 23), hạng mục 7 xong phần chính và 6 hạng mục đang làm (5, 8, 9, 10, 18, 21). Trong đó hạng mục 5, 9, 10 theo kế hoạch phải xong trong tuần 2 nhưng còn đang hoàn thiện (5 còn API Gateway, Lambda đăng ký và hủy job; 9 còn đổi worker sang Python 3.13; 10 chưa có ASG Spot), được làm tiếp đầu tuần 3. Hạng mục 13 (trang web người dùng) chưa bắt đầu, trễ so với kế hoạch tuần 2–3; 9 hạng mục còn lại chưa bắt đầu, đúng kế hoạch. Mốc M2 cuối tuần 3 giữ nguyên.

Các thông tin nền của Báo cáo 1 không đổi: đề tài đã chốt (minh chứng); tài khoản có quota 32 vCPU cho Spot và 16 vCPU cho On-Demand; nhóm dùng c7i-flex.large (2 vCPU, 4 GB) vì gói Free không cho dùng c7i.large; lưu trạng thái (checkpoint) và lập lịch theo hạn chót để lại làm hướng mở rộng.

Kế hoạch có thêm 3 hạng mục: hợp đồng giao diện (22) và Lambda xử lý DLQ (23) phát sinh và đã xong trong tuần 1–2; S3 bucket web và CloudFront cho cổng web (24) thêm vào tuần 3. Kế hoạch nay gồm 24 hạng mục, tổng 165 + 4 + 2 + 2 = 173 giờ-người; cộng 15% dự phòng thành khoảng 199 giờ-người, tức khoảng 13 giờ mỗi người mỗi tuần. Chia theo người (đã gồm dự phòng, làm tròn): TV1 (nhóm trưởng) ≈ 75 giờ, TV2 ≈ 67 giờ, TV3 ≈ 57 giờ. So với Báo cáo 1, TV1 nhận thêm hạng mục 22 và 24, TV2 nhận thêm hạng mục 23, còn TV2 và TV3 đổi cho nhau hạng mục 13 và 15 (xem ghi chú dưới Bảng 3.1).

Nỗ lực được ước lượng theo từng hạng mục nhỏ (WBS), dựa trên số thành phần và giao diện mỗi hạng mục phải làm. Khâu ghép nối và sửa lỗi được tách riêng 12 giờ vì đây là khâu dễ phát sinh thêm thời gian nhất. Các hạng mục chủ yếu là cấu hình hoặc giao diện (3, 9, 10, 13, 15, 16) được ước lượng thấp hơn vì nhóm dùng các tài nguyên có sẵn. Cột “Thành phần kiến trúc” chỉ rõ hạng mục thuộc khối nào trong Hình 2.1 và dịch vụ nào trong Hình 2.2; TV1, TV2, TV3 là ba thành viên theo thứ tự ở trang bìa. Cột “Trạng thái” cập nhật đến ngày 25/10/2026.

3.1. Bảng hạng mục công việc

Bảng 3.1. Các hạng mục công việc và ước tính nỗ lực

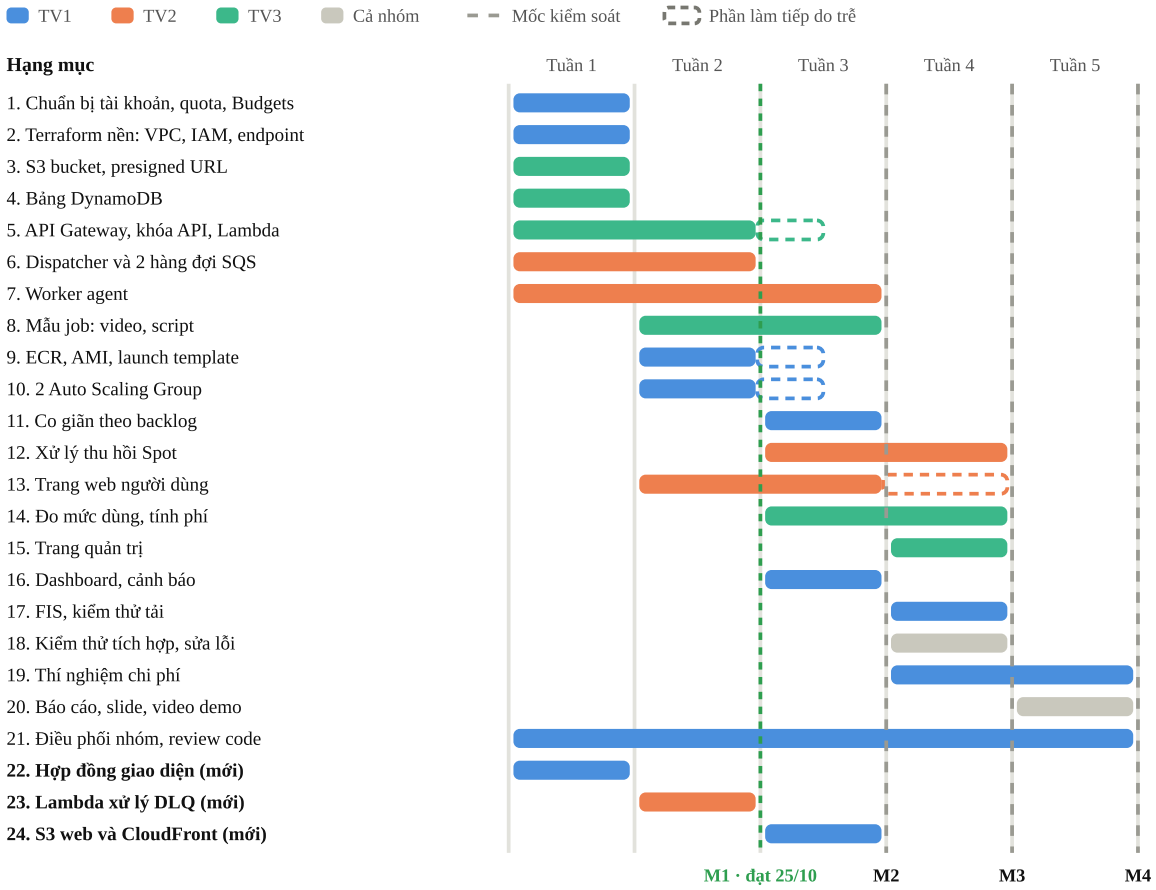
STT	Hạng mục công việc	Thành phần kiến trúc liên quan	Tuần	Nỗ lực (giờ-người)	Phụ trách	Trạng thái
1	Kiểm tra tài khoản: đặt AWS Budgets, xem quota Spot và On-Demand, bật thử máy Spot và On-Demand	Cụm worker (quota Spot, On-Demand); Giám sát và báo cáo chi phí (AWS Budgets)	1	1	TV1	Hoàn thành (minh chứng)
2	Terraform nền: VPC 3 AZ, subnet, security group, S3 và DynamoDB Gateway Endpoint, IAM role, lưu state	Mạng và bảo mật (Hình 2.2)	1	10	TV1	Hoàn thành
3	Tạo S3 bucket input, output, lifecycle rule xóa sau 7 ngày, hàm tạo presigned URL	Tiếp nhận job; Kết quả (Amazon S3)	1	4	TV3	Hoàn thành
4	Thiết kế bảng DynamoDB Users, Jobs, Usage, Tasks	Tài khoản và hạn mức; Đo mức dùng (DynamoDB)	1	5	TV3	Hoàn thành
5	API Gateway, Lambda authorizer kiểm tra khóa API, Lambda đăng ký và Lambda cho tạo job, xem trạng thái, hủy job (giao diện 2–6)	Lớp truy cập; Tài khoản và hạn mức	1–2	12	TV3	Đang làm (đã có Lambda tạo job, bấm chạy, xem trạng thái, authorizer; chưa có API Gateway, Lambda đăng ký và hủy job)
6	Lambda dispatcher tách job video thành từng video, 2 hàng đợi SQS và DLQ (giao diện 8–9)	Xếp job theo gói; Hàng đợi theo gói	1–2	6	TV2	Hoàn thành
7	Worker agent: long polling, giành job trên DynamoDB, gửi nhịp tim, chạy 2 suất song song trong container cách ly, gia hạn visibility, ghi kết quả và trạng thái (giao diện 10, 12–14)	Cụm worker (EC2, Docker)	1–3	22	TV2	Hoàn thành phần chính (phần Spot thuộc HM 12)
8	Hai mẫu job: nén video bằng FFmpeg (mỗi video một việc) và chạy script Python	Cụm worker; Phân mảnh tác vụ	2–3	6	TV3	Đang làm (mẫu script xong, mẫu video đang làm)

STT	Hạng mục công việc	Thành phần kiến trúc liên quan	Tuần	Nỗ lực (giờ-người)	Phụ trách	Trạng thái
9	Amazon ECR, danh sách image được phép, launch template và user data cài máy worker	Cụm worker; Cách ly người dùng	2	4	TV1	Đang làm (ECR, launch template xong; dùng user data thay Custom AMI; còn đổi worker sang Python 3.13)
10	2 Auto Scaling Group: On-Demand cho gói Ổn định; Spot nhiều AZ, price-capacity-optimized cho gói Tiết kiệm	Tự co giãn; Cụm worker	2	6	TV1	Đang làm (đã có 1 ASG máy On-Demand dùng để test luồng; còn ASG Spot và tách riêng 2 gói)
11	Co giãn theo backlog-per-instance bằng metric math, luật bật từ 0 máy, giữ 1 máy Ổn định trong giờ hoạt động (giao diện 15–16)	Tự co giãn (CloudWatch, Auto Scaling)	3	8	TV1	Chưa bắt đầu
12	Xử lý thu hồi Spot: bắt cảnh báo 2 phút, trả việc đang dở về hàng đợi, đếm số lần thu hồi, quá 2 lần thì chuyển sang On-Demand (giao diện 10–11)	Chịu lỗi khi Spot bị thu hồi	3–4	10	TV2	Chưa bắt đầu
13	Trang web người dùng: đăng ký, đăng nhập bằng khóa API, tạo job, chi tiết job, hóa đơn	Lớp truy cập (S3)	2–3	10	TV2	Chưa bắt đầu (trễ so với kế hoạch tuần 2–3)
14	Đo vCPU-giây, trừ credit, trần chi phí, nạp thêm, chính sách miễn phí job lỗi	Đo mức dùng và tính phí (Lambda billing, bảng Usage, giao diện 19)	3–4	10	TV3	Chưa bắt đầu
15	Trang quản trị: số máy, hàng đợi, doanh thu so với chi phí AWS	Lớp truy cập; Giám sát và báo cáo chi phí	4	5	TV3	Chưa bắt đầu
16	CloudWatch dashboard, cost allocation tag, cảnh báo	Giám sát và báo cáo chi phí	3	3	TV1	Chưa bắt đầu
17	Kịch bản AWS FIS thu hồi Spot và kiểm thử tải (giao diện 17)	Chịu lỗi khi Spot bị thu hồi; Tự co giãn	4	6	TV1	Chưa bắt đầu
18	Kiểm thử tích hợp toàn hệ thống và sửa lỗi	Cả ba lớp (Hình 2.1); giao diện 1–19	4	12	Cả nhóm	Đang làm (ghép M1 trên AWS ngày 25/10, minh chứng)

STT	Hạng mục công việc	Thành phần kiến trúc liên quan	Tuần	Nỗ lực (giờ-người)	Phụ trách	Trạng thái
19	Thí nghiệm chi phí: đo thực tế SpotPool so với EC2 On-Demand tự thuê trên hóa đơn AWS; so với Fargate và AWS Batch bằng tính toán từ bảng giá; đo thời gian chờ; tính điểm hòa vốn	Giám sát và báo cáo chi phí; Đo mức dùng	4–5	8	TV1	Chưa bắt đầu
20	Báo cáo cuối kỳ, slide, quay video demo dự phòng	Toàn bộ kiến trúc (Hình 2.1, 2.2): trình bày và demo	5	12	Cả nhóm	Chưa bắt đầu
21	Điều phối nhóm: họp tuần, theo dõi tiến độ, review code, ghép nhánh	Ghép nối các khối giữa ba thành viên (Hình 2.1, 2.2)	1–5	5	TV1	Đang làm
22	Hợp đồng giao diện giữa các thành phần (tài liệu `contracts.md`: tên tài nguyên, API, manifest, message, bảng, trạng thái)	Ghép nối các khối (Hình 2.1, 2.2)	1	4	TV1	Hoàn thành (minh chứng)
23	Lambda xử lý DLQ	Hàng đợi theo gói; DLQ (giao diện 18)	2	2	TV2	Hoàn thành
24	S3 bucket web và CloudFront cho cổng web (Terraform)	Lớp truy cập (S3, CloudFront)	3	2	TV1	Chưa bắt đầu
	Tổng			**173**		≈ 199 giờ-người kể cả 15% dự phòng

Ghi chú: Sau mốc M1, TV2 và TV3 đổi hai trang web để cân bằng khối lượng tuần 3–4: TV2 làm trang người dùng (HM 13), TV3 làm trang quản trị (HM 15) cùng với API /admin/overview; hai trang dùng chung một ứng dụng React. Mẫu script của HM 8 do TV2 làm xong trong tuần 2.

3.2. Tiến độ theo tuần



Hình 3.1. Biểu đồ Gantt kế hoạch 5 tuần

Các hạng mục được xếp để mỗi người làm từ 4 đến 17 giờ mỗi tuần, chưa tính dự phòng; tuần 5 chỉ còn đo chi phí, báo cáo và demo. So với Báo cáo 1, Hình 3.1 thêm ba hạng mục 22, 23, 24 và đổi màu hạng mục 13, 15 theo người phụ trách mới. Thanh liền là kế hoạch gốc; phần viền nét đứt là thời gian làm tiếp của các hạng mục đang trễ: 5, 9, 10 làm tiếp trong nửa đầu tuần 3, còn 13 kéo sang tuần 4 và vẫn xong trước M3. Bốn mốc kiểm soát, mỗi mốc lưu một bản chạy được:

- **M1, cuối tuần 2:** luồng chính chạy trọn vẹn từ nộp job đến tải kết quả. Đạt ngày 25/10, đúng hạn, chạy thực tế trên AWS bằng cách gọi trực tiếp các Lambda, chưa qua API Gateway; cổng web chưa có.
- **M2, cuối tuần 3:** tự co giãn, 2 gói và xử lý Spot bị thu hồi hoạt động. Giữ nguyên.
- **M3, cuối tuần 4:** đủ tính năng, dừng thêm tính năng mới.
- **M4, tuần 5:** nộp báo cáo và demo.

TÀI LIỆU THAM KHẢO

1. P. Mell, T. Grance. The NIST Definition of Cloud Computing, NIST SP 800-145, 2011.
2. AWS. Best practices for Amazon EC2 Spot.
3. AWS. Spot Instance interruption notices.
4. AWS. Scaling policy based on Amazon SQS.
5. AWS. Amazon SQS standard queues.
6. DoiT. AWS Public IPv4 Price Increase: The Complete Guide.
7. AWS. Amazon EC2 Spot Instance Advisor.
8. AWS. Track your Free Tier usage for Amazon EC2.
9. AWS. Restrict access to an Amazon S3 origin – Amazon CloudFront.
10. AWS. Website endpoints – Amazon S3.
11. AWS. Using dead-letter queues in Amazon SQS.
12. AWS. TransactWriteItems – Amazon DynamoDB API Reference.
13. AWS. DynamoDB Streams and AWS Lambda triggers.